

Ionirix Sovereign Update

Master Technical Architecture Document

Version 1.0 | April 18, 2026

Prepared for: Ionirix LLC — Founder, Technical Stakeholders, and Investors

Classification: Confidential — Internal & Investor Distribution Only

Author: Ionirix Architecture Office

NOTICE

This document describes the complete technical architecture for the Ionirix Sovereign Update. All numerical valuations, revenue projections, and financial estimates are intentionally omitted and require independent external appraisal. This document is intended for architectural evaluation and technical due diligence only.

1. Executive Summary

The Ionirix Sovereign Update is a transformative architectural initiative that elevates Ionirix from its current state into a **sovereign, event-driven world engine** — a system in which all world-state is authoritative, all simulation is deterministic, and all persistence is event-sourced. The update unifies a Python simulation engine with a TypeScript edge runtime, producing a system that is architecturally unique in the simulation-engine landscape.

Architectural Thesis: Ionirix will operate on four interlocking principles — authoritative world-state ownership through a Sovereign World Kernel, tick-driven deterministic simulation that guarantees reproducibility, event-sourced persistence that enables full temporal traversal and branching timelines, and a sovereign-industrial UI that communicates trust and precision to every user.

The system topology bridges two execution environments: a **Python Simulation Engine** for high-fidelity scientific computation and an **edge-native TypeScript**

Runtime (Cloudflare Workers + Durable Objects) for real-time state management, WebSocket subscriptions, and UI delivery. The **World-State Bus** — implemented as a Durable Object state bridge — mediates between these two environments with single-threaded consistency guarantees.

Core Deliverables

Deliverable	Description
1. Sovereign World Kernel	Deterministic, tick-based simulation kernel serving as the authoritative source of truth for all world-state. Manages agent orchestration, event routing, and anomaly detection.
2. World-State Bus	Python ↔ TypeScript integration layer built on Cloudflare Durable Objects. Provides schema-validated state bridging, WebSocket pub/sub, and a command layer with full causality tracing.
3. UI Overhaul	Sovereign-industrial design system with simulation dashboard, agent density maps, timeline scrubber, tick inspector, and scenario builder.
4. Chat Runtime	Simulation-aware conversational interface with world-state queries, agent introspection, narrative generation, and anomaly explanation.
5. Persistence Layer	Tiered persistence with world snapshots, agent memory versioning, append-only event logs, point-in-time rollback with timeline branching, and cross-mode continuity.
6. Valuation-Ready Documentation	Architecture diagrams, competitive positioning, IP surface analysis, and technical differentiation documentation. No fabricated valuations — architecture and moats only.

2. Architectural Overview

2.1 System Topology

The Ionirix Sovereign Update establishes a four-layer system topology. Each layer is independently deployable, horizontally addressable, and communicates exclusively through defined protocol boundaries.

#	Layer	Runtime	Responsibility
1	Python Simulation Engine	Python 3.12+	Scientific simulation compute. Executes the Sovereign World Kernel, advances ticks, runs agent logic, detects anomalies. Writes authoritative state to the World-State Bus.
2	World-State Bus	CF Durable Objects	Canonical state bridge. Accepts validated state from the Python engine, versions it, persists it to SQLite, and fans out to subscribers via WebSocket hibernation. Single-threaded consistency guarantee.
3	TypeScript Runtime	CF Workers	Edge-native request handling. Serves the Kernel Query API, processes commands, manages chat runtime, and delivers the UI. Reads state from Durable Object

#	Layer	Runtime	Responsibility
			subscriptions.
4	Sovereign UI	React + TypeScript	Sovereign-industrial interface. Renders simulation dashboard, agent density maps, timeline scrubber, tick inspector, and scenario builder. Zustand stores connected to World-State Bus WebSocket subscriptions.

Data flow: Python Simulation Engine → (state writes) → World-State Bus (Durable Object) → (WebSocket subscriptions) → TypeScript Runtime + Sovereign UI.
Commands flow in reverse: UI → TypeScript Runtime → Command Layer → World-State Bus → Python Engine.

2.2 Design Principles

Principle	Definition
Sovereignty-First	The system owns its state absolutely. No external dependency can corrupt, override, or bypass the kernel's authority over world-state. Every mutation is validated, versioned, and auditable.
Event-Driven	All state transitions are expressed as immutable events. The system reacts to events rather than polling for changes, enabling loose coupling, real-time responsiveness, and natural simulation semantics.
Deterministic Simulation	Given identical initial state and event sequence, the kernel produces byte-identical output. This enables reproducibility, rollback, branching timelines, and certification-grade

Principle	Definition
	auditability.
Edge-Native	The TypeScript runtime deploys to Cloudflare's global edge network. State management via Durable Objects eliminates traditional infrastructure. Latency is minimized by proximity to consumers.
Zero-Trust State	No component trusts state from another component without validation. Every state write passes through schema validation, consistency checks, and version verification before acceptance.

2.3 Component Registry

Module	Language / Runtime	Responsibility
Tick Manager	Python	Advances simulation time, schedules agent actions, enforces tick ordering
Event Router	Python	Receives, classifies, prioritizes, and routes events to handlers
Agent Orchestrator	Python	Manages agent lifecycle: spawn, execute, suspend, terminate
World-State Object	Python	Authoritative, versioned, immutable-per-tick state container
Anomaly Detector	Python	Pattern-matching engine flagging simulation trajectory deviations
State Bridge (DO)	TypeScript / CF DO	Canonical state bridge with WebSocket hibernation and SQLite persistence
Subscription Manager	TypeScript / CF DO	WebSocket pub/sub fanout with channel

Module	Language / Runtime	Responsibility
		taxonomy and filtering
Command Layer	TypeScript / CF Worker	Command validation pipeline: Auth → Schema → Consistency → Execute → Ack
Kernel Query API	TypeScript / CF Worker	External query interface for world-state, agents, timelines, anomalies
Chat Runtime	TypeScript / CF Worker	Simulation-aware conversational engine with NL-to-query translation
Narrative Engine	TypeScript / CF Worker	Event-to-prose pipeline with chronicle, dramatic, and analytical modes
Persistence Manager	TypeScript / CF Worker + R2	World snapshots, agent memory, event logs, tiered retention
Rollback Controller	Python + TypeScript	Point-in-time rollback with timeline branching
Sovereign UI	React + TypeScript	Simulation dashboard, density maps, timeline scrubber, tick inspector, scenario builder
Zustand State Stores	TypeScript	Client-side state management connected to World-State Bus WebSocket subscriptions

3. Sovereign World Kernel

3.1 Architecture

The Sovereign World Kernel is the authoritative source of truth for all world-state within the Ionirix system. It executes as a deterministic, tick-based simulation engine written in Python, where every state transition is reproducible and every mutation is versioned. No external system may write world-state directly — all mutations flow through the kernel's event processing pipeline.

The kernel follows a **deterministic tick-based execution model** inspired by discrete-event simulation (DES) frameworks, but adapted for agent-based world simulation. Each tick represents an atomic unit of simulation time. Within a tick, all events are collected, ordered deterministically, processed sequentially, and the resulting state is committed as an immutable snapshot before the next tick begins. This guarantees that replaying the same event sequence from the same initial state produces byte-identical results — a property essential for rollback, branching timelines, and debugging.

Kernel Lifecycle

Each tick follows a six-phase lifecycle:

#	Phase	Description
1	Init	Kernel initializes runtime, loads configuration, registers modules, sets tick counter to 0 or to the last persisted tick on recovery.
2	Load	Loads world-state from the most recent snapshot or constructs initial state from seed parameters. Validates state integrity via hash comparison.
3	Tick	Advances simulation time by one tick. Collects all pending events from the Event Router's inbound queue. Orders events deterministically by priority, then by source ID.
4	Evaluate	Processes each event sequentially. Agent Orchestrator executes agent actions. Anomaly Detector evaluates trajectory deviations. All state mutations are staged (not yet committed).

#	Phase	Description
5	Persist	Commits staged mutations to the World-State Object as an immutable tick snapshot. Computes <code>world_state_hash</code> . Writes delta to World-State Bus. Optionally writes full snapshot to persistence layer.
6	Idle	Kernel awaits next tick trigger (time-based or manual advance). During idle, external queries are served from the last committed state.

Module Decomposition

Tick Manager: The Tick Manager is responsible for advancing simulation time. It maintains the global tick counter, enforces minimum and maximum tick rates, and schedules agent actions within each tick. Agent action ordering within a tick is deterministic — agents are processed in a stable sort order defined by their spawn sequence and agent ID. The Tick Manager also supports manual tick advancement for debugging and scenario exploration.

Event Router: The Event Router receives events from all sources — agent actions, environment triggers, external command injections, and system-generated events (e.g., anomaly alerts). Each event is classified by type, assigned a priority, and placed in the tick's processing queue. The router enforces a strict event taxonomy (see Section 4.1) and rejects events that fail schema validation. Events are immutable after creation.

Agent Orchestrator: The Agent Orchestrator manages the full agent lifecycle: spawn, execute, suspend, and terminate. It distributes compute across agents within a tick according to the deterministic ordering provided by the Tick Manager. Agents execute within a sandboxed context — they can read the current world-state but can only mutate it by emitting events, which the Event Router processes in the next evaluation pass. The orchestrator also manages agent memory, goal stacks, and relationship graphs.

World-State Object: The World-State Object is the authoritative, versioned, immutable-per-tick state container. It holds the complete state of the simulation at any given tick: all agent states, environment properties, active effects, and metadata. Each tick produces a new version of the World-State Object with a cryptographic hash for integrity verification. Previous versions are retained for rollback and timeline branching. The object is structured as a nested dictionary with typed fields conforming to the schemas defined in Section 4.1.

Anomaly Detector: The Anomaly Detector is a pattern-matching engine that evaluates each tick's state transitions against expected simulation trajectories. It detects deviations such as agents with impossible state combinations, events that violate causal constraints, population metrics outside expected bounds, and emergent patterns that exceed configured thresholds. Detected anomalies are emitted as events with severity levels (info, warning, critical) and are routed to both the persistence layer and the UI's anomaly alert strip.

3.2 APIs

Kernel Control API (Internal)

The Kernel Control API is used internally by the simulation engine and by authorized command-layer calls. It is not exposed directly to external consumers.

Method	Parameters	Response
advance_tick()	count: int (default 1)	Returns {tick: int, hash: str, events_processed: int, duration_ms: float}
inject_event()	event: EventSchema	Returns {event_id: str, queued_for_tick: int, status: "accepted" "rejected"}
spawn_agent()	type: str, position: vec3, initial_state: dict, goals: list	Returns {agent_id: str, tick_created: int, status: "spawned"}
modify_environment()	env_id: str, mutations: dict	Returns {env_id: str, tick_modified: int, applied: list}
run_scenario()	scenario_id: str, params: dict, ticks:	Returns {scenario_id: str, branch_id: str,

Method	Parameters	Response
	int	ticks_executed: int, final_hash: str}
get_state_snapshot()	tick: int (optional, default current)	Returns {tick: int, hash: str, state: WorldStateObject, size_bytes: int}

Kernel Query API (External)

The Kernel Query API is exposed via the TypeScript Runtime (CF Workers) to the Sovereign UI and Chat Runtime. All queries are read-only and operate against the last committed tick state.

Method	Parameters	Response
query_world_state()	tick: int (optional), fields: list (optional filter)	Returns {tick: int, hash: str, state: partial WorldStateObject}
query_agent()	agent_id: str, tick: int (optional), include_memory: bool	Returns {agent: AgentSchema, history: AgentSchema[]} (if tick range)
query_timeline()	tick_start: int, tick_end: int, event_types: list (optional filter)	Returns {frames: FrameSchema[], event_count: int}
query_anomalies()	tick_range: [int, int], severity: str (optional), agent_id: str (optional)	Returns {anomalies: AnomalySchema[], count: int}

3.3 Lifecycle & State Machine

The kernel operates as a finite state machine with seven states. Transitions are triggered by explicit calls or system conditions, and each transition is guarded by preconditions that must be satisfied before the transition proceeds.

State	Transition To	Trigger	Guard Condition
Init	Loading	Kernel boot sequence	Configuration loaded and valid
Loading	Running	State loaded or constructed	World-state hash matches expected; all modules registered
Running	Running	advance_tick()	No pending rollback; event queue drained for current tick
Running	Paused	Manual pause or error threshold	Current tick fully committed
Paused	Running	Resume command	No unresolved critical anomalies
Running / Paused	Snapshot	Snapshot interval reached or manual trigger	Current tick fully committed
Snapshot	Running / Paused	Snapshot write confirmed	Snapshot hash verified against world-state
Running / Paused	Rollback	rollback_to_tick() command	Target tick has a valid snapshot; current state persisted
Rollback	Loading	Rollback initiated	Target snapshot loaded and hash-verified
Any	Terminated	Shutdown command or fatal error	Current state persisted; subscribers notified

STATE MACHINE INVARIANT

The kernel may never transition from **Running** to **Rollback** mid-tick. All tick processing must complete and commit before a rollback can begin. This preserves the immutability guarantee of committed tick states.

4. World-State Bus (Python ↔ TypeScript Integration)

4.1 Schema Definitions

All data flowing through the World-State Bus conforms to strict schemas. Schema validation occurs at the Durable Object boundary — any state write that fails validation is rejected with a structured error and logged to the audit trail.

Agent Schema

Field	Type	Required	Description
id	string (UUID)	Yes	Globally unique agent identifier, persistent across modes
type	string (enum)	Yes	Agent archetype classification (e.g., "explorer", "builder", "observer")
position	vec3 [x, y, z]	Yes	Current spatial coordinates in world-space
state	string (enum)	Yes	Current lifecycle state: active, suspended, terminated
memory	object	Yes	Structured memory: observations, learned patterns, short-term buffer
goals	array of object	Yes	Ordered goal stack with priority, status, and completion criteria

Field	Type	Required	Description
relationships	array of object	No	Connections to other agents: {target_id, type, strength, since_tick}
tick_created	integer	Yes	Tick number at which the agent was spawned
last_active	integer	Yes	Tick number of the agent's most recent action execution

Environment Schema

Field	Type	Required	Description
id	string (UUID)	Yes	Unique environment zone identifier
type	string (enum)	Yes	Zone classification (e.g., "terrain", "atmosphere", "structure")
properties	object	Yes	Key-value properties specific to the zone type
boundaries	object	Yes	Spatial boundaries: {min: vec3, max: vec3} or polygon definition
active_effects	array of object	No	Currently active environmental effects: {effect_type, magnitude, duration_ticks, source}
tick_modified	integer	Yes	Tick number of the most recent mutation to this

Field	Type	Required	Description
			zone

Event Schema

Field	Type	Required	Description
id	string (UUID)	Yes	Globally unique event identifier
type	string (enum)	Yes	Event classification (e.g., "agent_action", "env_change", "anomaly", "command")
source	string	Yes	Originating entity ID (agent, environment, system, or user)
target	string	No	Target entity ID, if applicable
payload	object	Yes	Event-type-specific data payload
tick	integer	Yes	Tick in which the event was generated
priority	integer (0-9)	Yes	Processing priority: 0 = highest (system), 9 = lowest (ambient)
causality_chain	array of string	No	Ordered list of ancestor event IDs that caused this event

Anomaly Schema

Field	Type	Required	Description
id	string (UUID)	Yes	Unique anomaly identifier
type	string (enum)	Yes	Anomaly classification (e.g., "state_violation", "trajectory_deviation", "population_outlier")
severity	string (enum)	Yes	Severity level: info, warning, critical
source_agents	array of string	No	Agent IDs involved in or affected by the anomaly
deviation_metric	float	Yes	Quantified deviation from expected trajectory (normalized 0.0–1.0)
tick_detected	integer	Yes	Tick at which the anomaly was detected
resolution_status	string (enum)	Yes	Status: open, acknowledged, resolved, escalated

Frame Schema

Field	Type	Required	Description
tick_number	integer	Yes	Sequential tick identifier
timestamp	ISO 8601 string	Yes	Wall-clock time at which the tick was processed
world_state_hash	string (SHA-256)	Yes	Cryptographic hash of the complete world-

Field	Type	Required	Description
			state at this tick
agent_count	integer	Yes	Number of active agents at tick completion
event_count	integer	Yes	Number of events processed during this tick
anomaly_count	integer	Yes	Number of anomalies detected during this tick
delta_from_previous	object	No	Compressed diff of state changes from the previous tick

4.2 State Bridge Implementation

Primary: Cloudflare Durable Object

The canonical state bridge is implemented as a Cloudflare Durable Object. This choice is architecturally significant: Durable Objects provide **single-threaded execution**, which eliminates the need for distributed locks or consensus protocols when managing world-state. The Durable Object is the single point of consistency between the Python simulation engine and the TypeScript runtime.

Key capabilities leveraged:

- **Single-threaded consistency:** All reads and writes to the state bridge are serialized within the Durable Object's execution context. No race conditions. No distributed consensus overhead.
- **WebSocket Hibernation API:** The Durable Object accepts WebSocket connections from UI clients and runtime consumers. During periods of inactivity, the Durable Object hibernates — releasing memory while keeping WebSocket connections alive at the Cloudflare network layer. When a new state write arrives, the object wakes, processes the update, and fans out to

all connected subscribers. This dramatically reduces compute costs during idle periods.

- **Embedded SQLite storage:** Each Durable Object has access to a private, transactional SQLite database. Tick-level state is persisted to SQLite for fast recovery and local querying. This provides the "warm" tier of the persistence model.
- **Global uniqueness:** Each Durable Object instance is globally unique and addressable by a stable ID. The World-State Bus uses a single Durable Object instance per simulation, ensuring that all state flows through a single authoritative point.

Fallback: Redis (Upstash)

For high-throughput read scenarios — particularly when multiple UI clients require rapid polling of agent state or when external analytics systems need to query state without placing load on the Durable Object — an Upstash Redis instance serves as a read replica. The Durable Object writes state updates to Redis asynchronously after each tick commit. Redis serves read-only queries with eventual consistency (typically <50ms lag behind the Durable Object).

Bridge Protocol

Step	Actor	Action
1	Python Sim Engine	Completes tick evaluation. Serializes world-state delta and frame metadata as JSON. Sends HTTP POST to Durable Object endpoint with authentication token.
2	Durable Object	Receives state write. Validates against schema definitions (Section 4.1). Verifies tick sequence (must be exactly <code>previous_tick + 1</code>). Checks <code>world_state_hash</code> integrity.

Step	Actor	Action
3	Durable Object	On validation success: persists to SQLite, increments internal version counter, updates in-memory state representation.
4	Durable Object	Fans out state update to all subscribed WebSocket connections, filtered by each subscriber's channel subscriptions and filters.
5	Durable Object	Asynchronously writes to Redis read replica (if configured). Returns acknowledgment to Python engine with {tick, version, status}.
6	Durable Object	If no further events arrive within the hibernation threshold, the Durable Object hibernates. WebSocket connections remain alive at the network layer.

4.3 Subscription Model

The World-State Bus implements a WebSocket-based publish/subscribe model from the Durable Object to all consuming clients: the Sovereign UI, the Chat Runtime, external analytics, and any authorized third-party consumer.

Channel Taxonomy

Channel	Update Frequency	Payload
world_state	Every tick	Frame metadata + full or delta world-state depending on subscriber preference
agent_updates	Per-agent, per-tick	Agent state changes, filtered by subscriber's agent ID or type filters

Channel	Update Frequency	Payload
events	Per-event	Individual events as they are committed, filterable by event type
anomalies	On detection	Anomaly objects as detected, filterable by severity
tick_metadata	Every tick	Lightweight frame metadata only (tick number, hash, counts) — low-bandwidth channel for status monitoring

Subscription Filtering

Subscribers can attach filters to their subscriptions to receive only relevant updates:

- **Agent ID filter:** Receive updates only for specified agent IDs
- **Event type filter:** Receive only events of specified types
- **Spatial region filter:** Receive updates only for entities within a defined bounding box
- **Severity filter:** Receive anomalies only at or above a specified severity level

Backpressure & Reconnection

If a subscriber cannot consume updates at the rate they are produced, the Durable Object implements a backpressure protocol: it buffers up to 100 messages per subscriber, then begins dropping the oldest unacknowledged messages (with a `dropped_count` field in the next delivered message so the subscriber knows to request a full state sync). On WebSocket disconnection, subscribers reconnect and send their last-known tick number; the Durable Object responds with a catch-up payload containing all state changes since that tick.

4.4 Command Layer

The Command Layer accepts mutation requests from the UI, Chat Runtime, and external API consumers. All commands are validated through a five-stage pipeline before execution.

Available Commands

Command	Description
inject_event	Inserts an external event into the kernel's event queue for processing in the next tick
spawn_agent	Creates a new agent with specified type, position, initial state, and goals
modify_environment	Applies mutations to a specified environment zone's properties or effects
run_scenario	Initiates a branching scenario with specified parameters and tick count
advance_tick	Manually advances the simulation by a specified number of ticks
pause_simulation	Pauses the simulation after the current tick completes
rollback_to_tick	Initiates a rollback to a specified tick number (must have a valid snapshot)

Validation Pipeline

Stage	Name	Action
1	Auth	Verify the caller's identity and authorization level. Commands are tiered: read (anyone), write (authenticated), admin (elevated).
2	Schema	Validate the command payload against the command's schema definition. Reject malformed requests with structured error.
3	Consistency	Verify that the command is consistent with current world-state. E.g., modify_environment must reference an existing

Stage	Name	Action
		env_id; rollback_to_tick must reference a tick with a valid snapshot.
4	Execute	Forward the validated command to the appropriate kernel method or Durable Object handler. Await execution result.
5	Acknowledge	Return a structured acknowledgment to the caller with command_id, status, result payload, and execution duration.

Command Audit Log: Every command that passes the Auth stage is logged to an append-only audit log with: command_id, caller_id, timestamp, command_type, payload hash, validation result, execution result, and full causality chain (which events were generated as a result of the command). This log is immutable and queryable for compliance and debugging.

5. UI Overhaul (Sovereign-Industrial)

5.1 Design System

Color Palette

Token	Hex Value	Name	Usage
--accent-primary	#00E5FF	Spectral Cyan	Primary accent, active states, selection highlights, interactive elements
--accent-	#1A73E8	Ion Blue	Secondary accent,

Token	Hex Value	Name	Usage
secondary			links, status indicators, chart lines
--surface-primary	#F5F7FA	Quantum White	Primary surface backgrounds, card panels, content areas
--surface-base	#0D1117	Pine Black	Base background, panel frames, text on light surfaces
--alert	#FFB300	Amber	Warnings, anomaly alerts, attention-required states
--critical	#FF1744	Signal Red	Critical anomalies, error states, destructive actions
--success	#00E676	Lattice Green	Success confirmations, healthy states, positive deltas

Typography

Context	Font	Usage
Data displays	Monospace (system)	Tick counters, agent IDs, hash values, JSON payloads, coordinates
Labels & headings	Geometric sans-serif	Panel titles, section headers, navigation labels, button text
Body text	System sans-serif	Descriptions, tooltips, chat messages, narrative prose

Layout Philosophy: Sovereign Paneling

The UI is composed of modular, dockable panels arranged in a sovereign paneling grid. Each panel is a self-contained functional unit with a translucent glass-

morphism border effect. Panels can be rearranged, resized, minimized, and pinned. The layout adapts to screen size while maintaining panel proportions. No decorative animation — all motion is purposeful (state transition indicators, loading states, data updates).

5.2 Core Views

Simulation Dashboard

The primary operational view. Displays real-time world-state at a glance:

- **Tick counter:** Large monospace display showing current tick number, ticks per second rate, and elapsed simulation time
- **Agent population metrics:** Total active agents, agents by type (bar), agents spawned/terminated this session
- **Event throughput gauge:** Events processed per tick (rolling average), peak, and current
- **Anomaly alert strip:** Horizontal bar at the top showing active anomalies by severity with expandable detail cards
- **World-state hash:** Truncated SHA-256 of the current world-state for integrity verification

Agent Density Maps

Spatial heatmap visualization of agent distribution across the simulation world.

Rendered via Canvas/WebGL for performance with large agent populations.

Features:

- Filter by agent type, state (active/suspended), and activity level (last N ticks)
- Adjustable resolution — from high-level continental view to individual agent dots
- Overlay modes: density heat, relationship graphs, movement trails
- Click-to-inspect: selecting a region or agent opens the Agent Inspector panel

Timeline Scrubber

Horizontal timeline visualization spanning the full simulation history. Tick marks indicate individual ticks; event markers (colored dots) indicate significant events; anomaly flags (amber/red triangles) indicate detected anomalies. Interaction model:

- **Drag-to-seek:** Scrub to any tick to load that tick's world-state into all panels
- **Range selection:** Select a tick range for replay, comparison, or export
- **Zoom:** Zoom in to individual ticks or out to see thousands of ticks with aggregated markers
- **Branch indicators:** If timeline branching has occurred, branch points are marked and selectable

Tick Inspector

Deep-dive view for a single tick. Shows the complete state transition that occurred during that tick:

- **Events processed:** Ordered list of all events with type, source, target, and payload summary
- **Agent state changes:** Diff view showing before/after state for each agent that changed
- **Environment mutations:** List of environment zones modified with property diffs
- **Anomalies detected:** Any anomalies flagged during this tick, with severity and description
- **Causality graph:** Visual representation of which events caused which state changes

Scenario Builder

Variable control panel for designing and executing simulation scenarios:

- **Variable controls:** Sliders, toggles, and seed inputs for adjustable simulation parameters

- **Scenario presets:** Named presets that configure multiple variables at once (e.g., "High Population Stress", "Resource Scarcity", "Peaceful Equilibrium")
- **Branch execution:** Run a scenario as a timeline branch without affecting the main simulation
- **Diff view:** Side-by-side comparison of two scenario branches at the same tick offset, highlighting divergent outcomes

5.3 Component Architecture

Layer	Technology	Responsibility
Component Framework	React 18+ / TypeScript	Component tree rendering, lifecycle management, event handling
State Management	Zustand	Client-side stores connected to World-State Bus WebSocket subscriptions. Stores: worldStateStore, agentStore, eventStore, anomalyStore, uiStore
Data Visualization	Canvas / WebGL	Heatmaps, density maps, movement trails, large-dataset rendering
Panel System	Standard DOM	Dockable panel containers, control elements, tables, lists, text
Layout System	Sovereign Paneling Grid	Responsive panel arrangement with minimum sizes, snap-to-grid, and panel state persistence

6. Chat Runtime Completion

6.1 Simulation-Aware Chat

The Ionirix Chat Runtime is not a generic conversational engine — it is a **simulation-aware interface** that receives world-state context on every message exchange. When a user sends a message, the chat engine automatically injects current simulation context into its processing pipeline:

Context Field	Content
Current tick	Tick number, elapsed simulation time, ticks per second
Active agents summary	Total count, agents by type, recently spawned/terminated
Recent events	Last 10 events with type, source, and summary
Active anomalies	All open anomalies with severity, type, and affected agents
Simulation state	Running, paused, or rollback-in-progress

The chat engine can also **issue simulation commands** via natural language. When the user says "advance 50 ticks" or "pause the simulation," the chat runtime translates the intent to the appropriate command (e.g., `advance_tick(count=50)`, `pause_simulation()`) and routes it through the Command Layer's full validation pipeline.

6.2 World-State Queries

The chat runtime includes a natural-language-to-structured-query translator. Users can ask questions about the simulation in plain language, and the chat engine converts them to Kernel Query API calls.

Natural Language Input	Translated API Call
"What is Agent-7 doing?"	<code>query_agent(agent_id="7",</code>

Natural Language Input	Translated API Call
	<code>include_memory=true)</code>
"Show me anomalies in the last 100 ticks"	<code>query_anomalies(tick_range=[current-100, current])</code>
"How many agents are active?"	<code>query_world_state(fields=["agent_count"])</code>
"What happened at tick 5000?"	<code>query_timeline(tick_start=5000, tick_end=5000)</code>
"Show me all critical anomalies"	<code>query_anomalies(severity="critical")</code>
"Compare Agent-3 and Agent-12"	<code>query_agent(agent_id="3") + query_agent(agent_id="12")</code>

Query results are formatted with inline visualizations where appropriate — agent states displayed as structured cards, timelines rendered as compact inline scrubbers, and anomaly lists displayed with severity-coded indicators.

6.3 Agent Introspection

The Chat Runtime provides deep inspection capabilities for individual agents:

- **State inspection:** Full read-out of an agent's current goals (ordered by priority), memory contents (observations, learned patterns), relationship graph (connections to other agents with type and strength), and complete action history.
- **Conversational agent interview:** The chat can "talk to" an agent — the user addresses an agent by ID, and the chat runtime generates responses from the agent's perspective, drawing on the agent's memory model, personality parameters, and current goal state. Responses are clearly labeled as generated-from-model, not claimed to be sentient.
- **Agent comparison:** Side-by-side comparison of two or more agents across all state dimensions: goals alignment, memory overlap, relationship graph intersection, behavioral trajectory similarity.

6.4 Narrative Generation

The Narrative Engine transforms raw simulation events into readable prose through a four-stage pipeline:

Stage	Name	Description
1	Event Extraction	Collects raw events from a specified tick range. Filters by significance threshold to exclude ambient/routine events.
2	Causal Chain Assembly	Traces causality_chain fields to construct causal graphs. Identifies root causes, cascading effects, and terminal outcomes.
3	Template Selection	Selects narrative template based on the chosen mode and the structure of the causal graph (linear, branching, converging).
4	Prose Generation	Generates natural-language narrative using the selected template, populated with entity names, actions, and outcomes from the causal graph.

Narrative Modes

Mode	Tone	Focus
Chronicle	Factual, neutral	Sequential account of what happened. "At tick 4521, Agent-7 encountered Agent-12 at position [34, 12, 0]. Agent-7 initiated a resource exchange..."
Dramatic	Conflict-focused	Emphasizes tensions,

Mode	Tone	Focus
		turning points, and resolutions. Highlights agent conflicts, goal clashes, and anomaly crises.
Analytical	Pattern-focused	Identifies emergent patterns, statistical trends, and systemic behaviors. "Over the last 500 ticks, explorer-type agents have converged on the northern quadrant at a rate 3.2× the expected baseline..."

Narratives can be exported as structured documents with section headings, entity references, and tick citations.

6.5 Anomaly Explanation & Predictive Reasoning

When the Anomaly Detector flags a deviation, the Chat Runtime automatically generates a structured analysis:

- **Root cause analysis:** Traces the anomaly's causality chain backward to identify the originating event or state condition. Presents the chain as a step-by-step narrative: "This anomaly originated from Event X at tick Y, which caused Agent Z to enter state W, which cascaded to..."
- **Impact assessment:** Identifies all agents affected by the anomaly (directly and transitively), all environment zones impacted, and all cascading events generated. Quantifies the scope as a percentage of the total agent population and world-state surface area.
- **Predictive projection:** If the anomaly's causal pattern continues unresolved, the system projects the likely state at tick + N (configurable). Projections include: expected agent population impact, environment degradation trajectory, and probability of secondary anomalies.

Users can ask follow-up questions about any anomaly in natural language. The chat engine maintains anomaly context across the conversation, allowing deep exploration without re-specifying the anomaly ID.

7. Persistence Layer

7.1 World Snapshots

The persistence layer captures full world-state snapshots at configurable tick intervals. Snapshots serve as restore points for rollback, branching, and recovery.

Property	Specification
Format	Compressed JSON (gzip). Full serialization of the World-State Object including all agent states, environment zones, active effects, and metadata.
Integrity	Each snapshot includes a SHA-256 <code>world_state_hash</code> computed over the canonical serialization. Snapshot is verified on write and on restore.
Interval	Configurable: default every 100 ticks. Can be adjusted per simulation or forced manually via <code>get_state_snapshot()</code> .
Storage	Cloudflare R2 object storage. Low-cost, S3-compatible, globally replicated. Each snapshot stored with key: <code>{sim_id}/snapshots/{tick_number}.json.gz</code>
Index	Snapshot index maintained in Durable Object SQLite: <code>tick_number</code> , <code>timestamp</code> , <code>hash</code> , <code>size_bytes</code> , <code>r2_key</code> , <code>branch_id</code> .

7.2 Agent Memory Persistence

Agent memory is a first-class persistent entity. Each agent's memory is stored as a structured document containing:

- **Goals:** Ordered goal stack with priority, status, completion criteria, and progress metrics
- **Observations:** Timestamped observations of other agents, environments, and events
- **Relationships:** Graph of connections to other agents with type, strength, and history
- **Learned patterns:** Behavioral patterns extracted from repeated observations

Memory is versioned per tick for rollback compatibility. When a rollback occurs, each agent's memory is restored to the version corresponding to the target tick. For inactive agents (no action in the last N ticks), memory is archived to cold storage (R2) and evicted from the Durable Object's active memory. Re-activation triggers a cold storage fetch.

7.3 Event Logs

All events are persisted to an append-only event log with full causality chain information. The log is the canonical record of everything that has ever happened in the simulation.

Property	Specification
Structure	Append-only. Events are immutable once written. Each entry contains the full EventSchema plus write_timestamp and log_sequence_number.
Indexing	Indexed by: tick, event_type, source_agent, target_agent. Supports compound queries (e.g., all events of type "agent_action" from Agent-7 in ticks 1000-2000).
Query interface	Queryable via the Kernel Query API (query_timeline()) and via the Chat Runtime's natural language interface.
Retention tiers	See tiered retention table below.

Tiered Retention Policy

Tier	Storage	Retention Window	Access Latency
Hot	DO SQLite	Last 10,000 ticks	<10ms (in-memory or local SQLite)
Warm	Cloudflare R2	Full simulation history	50-200ms (object fetch)
Cold	R2 Infrequent Access	Archived simulations	200-500ms (retrieval from cold tier)

7.4 Rollback System

The rollback system enables point-in-time recovery to any tick that has a valid snapshot. Rollback is a first-class operation — not a disaster recovery mechanism, but a core simulation capability used for branching, scenario comparison, and exploratory analysis.

Rollback Procedure

Step	Action
1	Validate: Verify target tick has a valid snapshot. Verify snapshot hash integrity. Persist current state as a snapshot (to preserve the pre-rollback timeline).
2	Load: Fetch snapshot from R2 (or SQLite if in hot tier). Decompress and deserialize. Verify hash against snapshot index.
3	Reset: Reset Durable Object in-memory state to the loaded snapshot. Reset tick counter to target tick. Clear event queue.
4	Replay (optional): If performing a partial rollback (rollback to tick X, then replay events from X to Y with

Step	Action
	modifications), replay events from the event log, applying them through the full kernel pipeline.
5	Notify: Broadcast state reset to all WebSocket subscribers with {type: "rollback", target_tick, branch_id, previous_tick}. Subscribers must discard cached state and re-sync from the new current state.

Timeline branching: Every rollback creates a new timeline branch. The original timeline is preserved — its snapshots and event logs remain in storage, addressable by branch_id. Users can switch between branches, compare them, or merge insights from one branch into another. The Scenario Builder UI provides visual branch management.

7.5 Cross-Mode Continuity

Ionirix supports three simulation scales, each representing a different level of world granularity:

Mode	Scale	Description
Planetary	Single-region, high fidelity	Local simulation with maximum agent detail. All agents fully simulated with complete memory, goals, and relationships. Default mode.
Cosmic	Multi-region, medium fidelity	Multiple regions simulated concurrently. Agents in non-focused regions are simulated at reduced fidelity (statistical approximation). Inter-region interactions via event bridging.
Multiverse	Branching timelines, variable fidelity	Multiple timeline branches running concurrently. Enables scenario comparison at

Mode	Scale	Description
		scale. Shared agent identity across branches via universal agent ID.

State Migration Protocol

When transitioning between modes:

1. **Snapshot current state** in the origin mode as a full snapshot with mode transition metadata.
2. **Transform state** according to the target mode's fidelity requirements.
Planetary→Cosmic: compress non-focused agent states to statistical summaries. Cosmic→Planetary: expand statistical summaries to individual agents using seed-based generation.
3. **Verify agent identity:** All agents maintain their universal agent ID across mode transitions. Memory and relationship graphs are preserved at the target mode's fidelity level.
4. **Initialize target mode:** Load transformed state into the target mode's kernel instance. Resume tick execution.
5. **Notify subscribers:** Broadcast mode transition event to all subscribers with new mode, transformed state summary, and any agents that were compressed/expanded.

8. Development Plan

Phase	Timeline	Focus Area	Deliverables & Milestone
Phase 1	Weeks 1-4	Foundation	Deliverables: Sovereign World Kernel core, World-State Object with tick versioning, Tick

Phase	Timeline	Focus Area	Deliverables & Milestone
			Manager, basic Event Router, Kernel Control API (internal). Milestone: Kernel can advance ticks and route events with deterministic state. Given identical inputs, produces byte-identical output.
Phase 2	Weeks 5-8	Integration	Deliverables: World-State Bus (Durable Object bridge), schema validation layer, WebSocket subscription model, command layer with validation pipeline, Python simulation engine integration. Milestone: Python sim writes state, TS runtime reads it in real-time via WebSocket. Commands flow from TS to Python and execute successfully.
Phase 3	Weeks 9-12	Intelligence	Deliverables: Agent Orchestrator with full lifecycle, Anomaly Detector with pattern-matching, Chat Runtime with simulation awareness, world-state query engine, agent introspection system.

Phase	Timeline	Focus Area	Deliverables & Milestone
			Milestone: Agents run autonomously, anomalies are detected and reported, chat can query the world and inspect agents.
Phase 4	Weeks 13-16	Interface	Deliverables: Sovereign-industrial design system implementation, Simulation Dashboard, Agent Density Maps, Timeline Scrubber, Tick Inspector, Scenario Builder, Zustand stores wired to World-State Bus. Milestone: Full visual simulation control and monitoring. All five core views operational and connected to live data.
Phase 5	Weeks 17-20	Persistence & Scale	Deliverables: World snapshots to R2, agent memory persistence with versioning, event log with tiered retention, rollback system with timeline branching, cross-mode continuity protocol. Milestone: Full persistence layer operational. Rollback to any

Phase	Timeline	Focus Area	Deliverables & Milestone
			snapshotted tick works. Mode transitions between Planetary, Cosmic, and Multiverse succeed.
Phase 6	Weeks 21-24	Polish & Documentation	Deliverables: Narrative generation engine, anomaly explanation and predictive reasoning, architecture diagrams, technical documentation, release notes, investor-grade architecture summary. Milestone: Production-ready sovereign world engine with complete documentation suite. No fabricated valuations — architecture and differentiation only.

TIMELINE NOTE

The 24-week plan assumes a focused development team. Phases are sequential for clarity but allow overlap where dependencies permit (e.g., UI design system work can begin during Phase 3). Critical path runs through Phases 1→2→3, as each layer depends on the one below it.

9. Technical Justification

9.1 Why Event-Driven Architecture

Event-driven architecture (EDA) is the natural fit for a simulation engine. Simulations are inherently event-producing systems — agents act, environments change, anomalies emerge. EDA provides loose coupling between producers and consumers, enabling the Python simulation engine and the TypeScript runtime to evolve independently. Real-time responsiveness is built in: the World-State Bus pushes updates to subscribers as events occur, eliminating polling overhead. Event sourcing — storing the complete event log as the system of record — enables full temporal traversal, rollback, replay, and branching timelines. Microsoft's Azure Architecture Center identifies event sourcing as essential for systems requiring "auditability and historical reconstruction," which directly maps to Ionirix's core requirements.

9.2 Why Cloudflare Durable Objects

Durable Objects solve the hardest problem in the Ionirix architecture: consistent, low-latency state coordination between a Python backend and a globally distributed edge runtime. Key architectural advantages:

- **Single-threaded execution:** Eliminates distributed locks, consensus protocols, and race conditions. All state mutations are serialized within the Durable Object — the ideal model for an authoritative state bridge.
- **WebSocket Hibernation:** The Hibernation API allows the Durable Object to release memory during idle periods while keeping WebSocket connections alive at the Cloudflare network layer. When a state write arrives, the object wakes automatically. This provides real-time push semantics at a fraction of the cost of persistent compute.
- **Embedded SQLite:** Each Durable Object has access to a transactional SQLite database. This provides the "hot" persistence tier without requiring external database infrastructure.

- **Zero infrastructure management:** No servers to provision, scale, or maintain. The Durable Object scales with Cloudflare's global network. Deployment is a `wrangler deploy`.
- **Global uniqueness:** Each Durable Object instance is globally unique and addressable, providing a natural model for "one authoritative state bridge per simulation."

9.3 Why Deterministic Tick Model

Deterministic tick-based execution is non-negotiable for a sovereign world engine. The key properties it provides:

- **Reproducibility:** Given identical initial state and event sequence, the kernel produces byte-identical output. This is verified via cryptographic hashing of the world-state at each tick. Reproducibility is essential for debugging ("replay this exact scenario"), certification ("prove the simulation behaved correctly"), and scientific rigor.
- **Rollback capability:** Because every tick produces an immutable state snapshot, the system can roll back to any previous tick by loading that snapshot. Without determinism, rollback would require storing not just state but the exact execution environment — a much harder problem.
- **Branching timelines:** Deterministic execution enables meaningful branching. Two branches from the same snapshot with the same events will produce identical results. Two branches with different events will produce meaningfully different results — not noise from non-deterministic execution.

9.4 Why Python + TypeScript Split

The dual-language architecture is a deliberate optimization, not a compromise:

- **Python for simulation:** Python's scientific computing ecosystem (NumPy, SciPy, custom simulation libraries) is unmatched. Agent behavior models, anomaly detection algorithms, and world-state computations benefit from Python's expressiveness and library support. The simulation engine runs on

dedicated compute, where Python's performance characteristics are acceptable and its development velocity is advantageous.

- **TypeScript for edge runtime:** The TypeScript runtime deploys to Cloudflare Workers, executing at the edge with sub-millisecond cold start times. TypeScript's type system provides compile-time safety for the complex schema validation and WebSocket management code. React + TypeScript is the standard stack for the Sovereign UI.
- **Clean boundary:** The World-State Bus provides a clean protocol boundary between the two languages. Each side communicates via JSON over HTTP/WebSocket with strict schema validation. Neither side needs to understand the other's implementation details.

9.5 Why Sovereign-Industrial UI

The sovereign-industrial aesthetic is an architectural decision with strategic implications:

- **Differentiation:** In a landscape of generic dashboards and material-design clones, the sovereign-industrial design language is immediately recognizable. It communicates that this is not a consumer toy — it is an instrument of precision.
- **Trust signaling:** The cold, precise aesthetic — monospace data displays, glass-panel framing, minimal animation — communicates reliability and rigor. Users interacting with complex simulation data need to trust the interface. Sovereign-industrial design earns that trust through visual discipline.
- **Functional optimization:** The design system is optimized for information density without clutter. Dockable panels, sovereign paneling grids, and purposeful motion keep the user focused on the data, not the decoration.

9.6 Architectural Synthesis

Ionirix synthesizes three established architectural patterns into a unified system:

Pattern	How Ionirix Uses It	Adaptation
Event Sourcing	All state transitions are recorded as immutable events in an append-only log. World-state can be reconstructed by replaying events from any snapshot.	Adapted for tick-based simulation: events are grouped by tick and processed deterministically within each tick, rather than processed individually in arrival order.
CQRS	Command and query responsibilities are strictly separated. Commands flow through the Command Layer to the kernel. Queries are served from the Durable Object's read-optimized state.	Extended with the World-State Bus: the Durable Object serves as both the write model (accepting state from the Python engine) and the read model (serving subscriptions to the TS runtime).
Actor Model	Agents are actor-like entities with encapsulated state, message-based communication (via events), and supervised lifecycle management (via the Agent Orchestrator).	Adapted for deterministic simulation: unlike traditional actor systems where message ordering is non-deterministic, Ionirix's agents process events in a deterministic order within each tick.

10. Valuation-Ready Summary

VALUATION NOTICE

All numerical valuations, revenue projections, and financial estimates require independent external appraisal and are intentionally omitted from this document. The following section describes architectural moats, technical differentiation, and IP surface area for evaluation purposes only.

10.1 Architectural Moats

Moat	Description
Deterministic World Kernel	The Sovereign World Kernel's deterministic tick-based execution model is architecturally unique among agent-based simulation platforms. Competitors that rely on non-deterministic scheduling cannot provide rollback, branching timelines, or reproducible simulation without fundamental re-architecture. This is a structural moat — it cannot be patched onto an existing non-deterministic system.
Edge-Native State Bridge	The Durable Object state bridge is a novel architectural pattern — using Cloudflare's single-threaded, globally unique Durable Objects as the canonical consistency point between a Python backend and a TypeScript edge runtime. This pattern eliminates entire categories of distributed systems complexity (consensus, distributed locks, split-brain scenarios) and scales with Cloudflare's global network.
Sovereign UI Design System	The sovereign-industrial design language is a proprietary aesthetic that creates immediate brand recognition and communicates premium positioning. The paneling system, color architecture, and interaction model are purpose-built for simulation control — not adapted from generic dashboard frameworks.
Cross-Mode Continuity	The ability to transition seamlessly between Planetary, Cosmic, and Multiverse simulation modes — with persistent agent identity and memory across modes — is architecturally unprecedented. This capability requires the deterministic kernel, event-sourced persistence, and the state migration protocol to work in concert.

10.2 Technical Differentiation

Ionirix differentiates from existing simulation platforms, game engines, and agent frameworks across several dimensions:

- **vs. traditional simulation engines:** Most simulation engines provide either deterministic execution *or* real-time interactivity. Ionirix provides both — deterministic tick-based simulation with real-time WebSocket push to a live UI.
- **vs. game engines:** Game engines optimize for frame-rate rendering. Ionirix optimizes for state-authority and temporal traversal. The kernel owns the state; the UI is a read-only consumer. This inversion of the typical game engine architecture enables rollback, branching, and reproducibility.
- **vs. agent frameworks:** Existing agent frameworks (e.g., LangGraph, CrewAI, AutoGen) focus on LLM orchestration. Ionirix provides a full world simulation layer — spatial positioning, environment modeling, anomaly detection, and cross-mode continuity — that agent frameworks do not address.
- **vs. event-sourcing platforms:** Event sourcing platforms (e.g., EventStoreDB, Axon) provide the persistence pattern but not the simulation engine. Ionirix integrates event sourcing as a persistence strategy within a complete simulation kernel.

10.3 IP Surface Area

IP Asset	Description
Kernel Architecture	Sovereign World Kernel design — tick lifecycle, deterministic execution model, module decomposition, state machine specification
Bus Protocol	World-State Bus protocol — Durable Object bridge pattern, schema validation pipeline, WebSocket subscription model with channel

IP Asset	Description
	taxonomy and filtering
UI Design System	Sovereign-industrial design language — color architecture, typography system, sovereign paneling layout, component library specifications
Persistence Model	Tiered persistence architecture — snapshot system, agent memory versioning, event log with causality chaining, timeline branching protocol
Cross-Mode Protocol	State migration between simulation modes — fidelity transformation algorithms, universal agent identity, mode transition handoff
Narrative Engine	Event-to-prose pipeline — causal chain assembly, template selection, multi-mode narrative generation
Anomaly System	Anomaly detection and explanation — pattern-matching engine, root cause analysis, impact assessment, predictive projection

10.4 Scalability Thesis

The Ionirix architecture is edge-native by design. The TypeScript runtime deploys to Cloudflare Workers, which execute in 300+ data centers worldwide. Durable Objects are automatically placed close to the primary consumer and can be migrated globally. As Cloudflare expands its network, Ionirix's edge runtime scales with it — no re-architecture required. The Python simulation engine can be deployed to any compute environment (cloud VMs, dedicated servers, or future Cloudflare Containers) and communicates with the edge runtime via the World-State Bus protocol.

This architecture enables a clean separation of scaling concerns: simulation compute scales vertically (more powerful Python instances), state management scales with Cloudflare's infrastructure (Durable Objects + R2), and UI delivery scales globally at the edge (Workers).

Appendix A: Glossary

Term	Definition
Tick	The atomic unit of simulation time. Each tick represents one discrete step in the simulation. All events within a tick are processed deterministically before the next tick begins.
Frame	A metadata record summarizing a single tick: tick number, timestamp, world-state hash, agent count, event count, anomaly count, and delta from the previous tick.
World-State	The complete, authoritative description of the simulation at a given tick. Includes all agent states, environment zones, active effects, and metadata. Versioned and immutable per tick.
Sovereign Kernel	The deterministic simulation engine that owns and governs all world-state. No external system may bypass its authority. "Sovereign" denotes absolute ownership of state truth.
World-State Bus	The integration layer between the Python simulation engine and the TypeScript edge runtime. Implemented as a Cloudflare Durable Object that accepts, validates, persists, and distributes world-state.
State Bridge	The Durable Object instance serving as the canonical consistency point between the Python and TypeScript runtimes. Synonym for the World-State Bus's core component.
Agent Orchestrator	The kernel module responsible for managing agent lifecycle (spawn, execute, suspend, terminate) and distributing compute within each tick.
Anomaly	A detected deviation from expected simulation trajectories. Classified by type and severity. Triggers automatic root cause analysis and predictive projection.

Term	Definition
Cross-Mode Continuity	The protocol enabling seamless state migration between Planetary, Cosmic, and Multiverse simulation modes while preserving agent identity, memory, and relationships.
Timeline Branch	A divergent simulation timeline created by rollback or scenario execution. The original timeline is preserved. Branches share a common ancestor tick.
Sovereign Paneling	The UI layout system composed of modular, dockable panels with glass-morphism borders. Each panel is a self-contained functional unit.
Causality Chain	An ordered list of ancestor event IDs tracing the causal lineage of an event. Used for root cause analysis, impact assessment, and narrative generation.
Hibernation	A Cloudflare Durable Object feature where the object releases memory during inactivity while WebSocket connections remain alive at the network layer. Reduces cost without disconnecting clients.
Deviation Metric	A normalized (0.0–1.0) quantification of how far an observed state deviates from the expected simulation trajectory. Used by the Anomaly Detector.

Appendix B: Schema Reference

Complete JSON schema definitions for all core entity types. These schemas are enforced at the World-State Bus boundary.

B.1 Agent Schema

Field	Type	Required	Constraints	Example
id	string	Yes	UUID v4 format	"a1b2c3d4- ..."

Field	Type	Required	Constraints	Example
type	string	Yes	enum: explorer, builder, observer, guardian, catalyst	"explorer"
position	array [3]	Yes	Three floats [x, y, z]	[34.5, 12.0, 0.0]
state	string	Yes	enum: active, suspended, terminated	"active"
memory	object	Yes	Keys: observations, patterns, short_term	{"observations": [...], ...}
goals	array	Yes	Each: {id, description, priority: 0-9, status, criteria}	[{"id": "g1", ...}]
relationships	array	No	Each: {target_id, type, strength: 0.0- 1.0, since_tick}	[{"target_id": : "...", ...}]
tick_created	integer	Yes	≥ 0	0
last_active	integer	Yes	$\geq$ tick_created	4521

B.2 Environment Schema

Field	Type	Required	Constraints	Example
id	string	Yes	UUID v4 format	"e1f2g3h4-..."
type	string	Yes	enum: terrain, atmosphere, structure, aquatic, void	"terrain"
properties	object	Yes	Type-specific	{"elevation":

Field	Type	Required	Constraints	Example
			key-value pairs	120, "biome": "forest"}
boundaries	object	Yes	{min: [x,y,z], max: [x,y,z]} or {polygon: [...points]}	{"min": [0,0,0], "max": [100,100,10]}
active_effects	array	No	Each: {effect_type, magnitude: float, duration_ticks: int, source: str}	[{"effect_type": "storm", ...}]
tick_modified	integer	Yes	≥ 0	3200

B.3 Event Schema

Field	Type	Required	Constraints	Example
id	string	Yes	UUID v4 format	"ev-a1b2c3-..."
type	string	Yes	enum: agent_action, env_change, anomaly, command, system	"agent_action"
source	string	Yes	Entity ID or "system"	"a1b2c3d4-..."
target	string	No	Entity ID or null	"e1f2g3h4-..."
payload	object	Yes	Type-specific data; validated per event type	{"action": "move", "delta": [1,0,0]}
tick	integer	Yes	≥ 0	4521
priority	integer	Yes	0-9 (0 = highest)	5
causality_cha	array	No	Ordered list of	["ev-x1...",

Field	Type	Required	Constraints	Example
in			ancestor event IDs	"ev-y2..."

B.4 Anomaly Schema

Field	Type	Required	Constraints	Example
id	string	Yes	UUID v4 format	"an-a1b2c3-..."
type	string	Yes	enum: state_violation, trajectory_deviation, population_outlier, causal_paradox	"trajectory_deviation"
severity	string	Yes	enum: info, warning, critical	"warning"
source_agents	array	No	Array of agent ID strings	["a1b2c3d4-..."]
deviation_metric	float	Yes	0.0-1.0 (normalized)	0.73
tick_detected	integer	Yes	≥ 0	4521
resolution_status	string	Yes	enum: open, acknowledged, resolved, escalated	"open"

B.5 Frame Schema

Field	Type	Required	Constraints	Example
tick_number	integer	Yes	Sequential, ≥ 0	4521

Field	Type	Required	Constraints	Example
timestamp	string	Yes	ISO 8601	"2026-04-18T23:00:00Z"
world_state_hash	string	Yes	SHA-256, 64 hex chars	"a3f8c2e1..."
agent_count	integer	Yes	≥ 0	1247
event_count	integer	Yes	≥ 0	342
anomaly_count	integer	Yes	≥ 0	2
delta_from_previous	object	No	Compressed state diff	{"agents_changed": 15, ...}

Appendix C: API Reference

C.1 Kernel Control API (Internal)

Internal API consumed by the Python simulation engine and authorized command-layer calls. Not exposed to external consumers.

Method	Path	Parameters	Response
POST	/kernel/tick/advance	count: int (default 1)	{tick, hash, events_processed, duration_ms}
POST	/kernel/event/inject	event: EventSchema (body)	{event_id, queued_for_tick, status}
POST	/kernel/agent/spawn	type, position, initial_state, goals (body)	{agent_id, tick_created, status}
PATCH	/kernel/env/{env_id}	mutations: object (body)	{env_id, tick_modified, applied}
POST	/kernel/scenario/run	scenario_id, params, ticks (body)	{scenario_id, branch_id, ticks_executed, final_hash}
GET	/kernel/state/snapshot	tick: int (query,	{tick, hash, state,

Method	Path	Parameters	Response
		optional)	size_bytes}
POST	/kernel/simulation/pause	None	{tick_paused_at, status: "paused"}
POST	/kernel/simulation/resume	None	{tick_resumed_at, status: "running"}
POST	/kernel/rollback	target_tick: int (body)	{target_tick, branch_id, previous_tick, status}

C.2 Kernel Query API (External)

External API served by the TypeScript Runtime (CF Workers). Available to the Sovereign UI, Chat Runtime, and authorized external consumers. All endpoints are read-only.

Method	Path	Parameters	Response
GET	/api/world-state	tick: int (optional), fields: string[] (optional)	{tick, hash, state}
GET	/api/agent/{agent_id}	tick: int (optional), include_memory: bool (default false)	{agent: AgentSchema}
GET	/api/agent/{agent_id}/history	tick_start, tick_end: int	{history: AgentSchema[]}
GET	/api/timeline	tick_start, tick_end: int, event_types: string[] (optional)	{frames: FrameSchema[], event_count}
GET	/api/anomalies	tick_start, tick_end: int, severity: string (optional), agent_id: string (optional)	{anomalies: AnomalySchema[], count}
GET	/api/events	tick_start, tick_end: int, type:	{events: EventSchema[],

Method	Path	Parameters	Response
		string (optional), source: string (optional)	count}
GET	/api/branches	None	{branches: [{id, parent_tick, created_at, tick_count}]}
GET	/api/snapshots	branch_id: string (optional)	{snapshots: [{tick, timestamp, hash, size_bytes}]}

C.3 WebSocket Subscription Protocol

Direction	Message Type	Payload
Client →	subscribe	{channels: string[], filters: {agent_ids?: string[], event_types?: string[], severity?: string, region?: {min: vec3, max: vec3}}}
Client →	unsubscribe	{channels: string[]}
Client →	sync	{last_known_tick: int} — requests catch-up from last known tick
→ Client	state_update	{channel: string, tick: int, data: object, dropped_count?: int}
→ Client	sync_response	{from_tick: int, to_tick: int, frames: FrameSchema[], full_state?: WorldStateObject}
→ Client	rollback_notice	{type: "rollback", target_tick: int, branch_id: string, previous_tick: int} — client must discard cached state
→ Client	error	{code: string, message: string, details?: object}

— End of Document —

Ionirix Sovereign Update — Master Technical Architecture v1.0 | April 18, 2026

Confidential — Ionirix LLC